End-to-end capstone for one endpoint

Session M6-02 | 3 hours

One slice

Implement and review:

```
POST /api/bookmarks
```

Everything else is deferred.

The result is a handoff

- issue;
- patch;
- test results;
- review decision and next action.

Four terms

- **Synthetic data**: made-up bookmarks, such as `https://docs.example.test/node`.
- **Specification ID**: the label of one requirement, such as `FR-007`.
- **Response envelope**: success is `{ "data": ... }`. Failure is `{ "error": { "code", "message" } }`.
- **Baseline**: test results before the change.

The scope lives in the issue

Implement only `POST /api/bookmarks`, use synthetic data, and keep the existing dependencies.

Session M5-02 is an input

The accepted specification must separate:

Behavior	Implementation constraint
Observable request and response rules	Node.js, Express, in-memory storage
Validation and duplicate behavior	Existing dependencies only
Acceptance scenarios	Exact files and test command

Issue trace

Every acceptance criterion maps to:

```
specification ID -> planned file -> executable test
```

If the issue adds behavior missing from the specification, fix the issue first.

Worked example: spec to issue to prompt

FR-007: a second request with the same URL returns `409`.

```
Issue: Reject duplicate bookmark URLs
Specification IDs: FR-007, FR-008
Criterion: repeated url -> 409, error.code CONFLICT
Test: "rejects a duplicate URL"
Files: src/routes/bookmarks.js, src/models/bookmark.js
```

```
Implement this issue. Existing dependencies only. Don't edit tests/.
List the files and spec IDs first, then change, then run npm test.
```

Scope

In scope:

- create one bookmark;
- validate the accepted fields;
- reject a duplicate URL;
- return the documented envelope and status.

Out of scope: authentication, persistence, other endpoints, deployment, and new packages.

The failing tests are the baseline

```
npm test
```

Health passes. The Bookmark tests fail because the route doesn't exist. They are the spec as executable checks. You're done when they pass. Don't edit them.

Review and handover

Passing tests are where review starts.

1. Run the suite.
2. Check the diff: no edits under `tests/`, no new dependency.
3. Name a test for each specification ID.
4. Break a rule on purpose. The matching test must fail.
5. Read the responses against the spec.
6. Record the decision: approve or request changes.

Lab time plan

Phase	Time
Review the specification	15 min
Write the issue and run the baseline	25 min
Prompt Copilot to implement it	40 min
Review, test, and hand off	40 min